A Taxonomy of Self-Admitted Technical Debt in Deep Learning Systems

Federica Pepe
University of Sannio
Benevento, Italy
f.pepe8@studenti.unisannio.it

Fiorella Zampetti
University of Sannio
Benevento, Italy
fiorella.zampetti@unisannio.it

Antonio Mastropaolo
William & Mary
Williamsburg, VA, United States
amastropaolo@wm.edu

Gabriele Bavota
Software Institute
Università della Svizzera Italiana
Lugano, Switzerland
gabriele.bavota@usi.ch

Massimiliano Di Penta
University of Sannio
Benevento, Italy
dipenta@unisannio.it

2024 IEEE International Conference on Software Maintenance and Evolution (ICSME) | 979-8-3503-9568-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICSME58944.2024.00043

***Abstract*—The development of Machine Learning (ML)- and, more recently, of Deep Learning (DL)-intensive systems requires suitable choices, *e.g.,* in terms of technology, algorithms, and hyper-parameters. Such choices depend on developers' experience, as well as on proper experimentation. Due to limited time availability, developers may adopt suboptimal, sometimes temporary choices, leading to a technical debt (TD) specifically related to the ML code. This paper empirically analyzes the presence of Self-Admitted Technical Debt (SATD) in DL systems. After selecting 100 open-source Python projects using popular DL frameworks, we identified SATD from their source comments and created a stratified sample of 443 SATD to analyze manually. We derived a taxonomy of DL-specific SATD through open coding, featuring seven categories and 41 leaves. The identified SATD categories pertain to different aspects of DL models, some of which are technological (*e.g.,* due to hardware or libraries) and some related to suboptimal choices in the DL process, model usage, or configuration. Our findings indicate that DL-specific SATD differs from DL bugs found in previous studies, as it typically pertains to suboptimal solutions rather than functional (*e.g.,* blocking) problems. Last but not least, we found that state-of-the-art static analysis tools do not help developers avoid such problems, and therefore, specific support is needed to cope with DL-specific SATD.**

***Index Terms*—Technical Debt, Deep Learning, Open-Source**

I. Introduction

In recent years, Deep Learning (DL) has emerged as a new force that, among other applications, has allowed developers to overcome several limitations of the existing "shallow" Machine Learning (ML) techniques, and has tackled problems and challenges that were previously infeasible, creating a real revolution in several industrial sectors. Indeed, DL has been widely used in several domains, such as finance, *e.g.,* fraud detection [3], healthcare to improve diagnosis and treatment of diseases, as well as patient outcomes [2], autonomous driving [10], and, last but not least, software engineering [46]. However, the adoption of DL can be cumbersome and can lead to, especially for business, mission, and safety-critical systems, software bugs producing huge economic losses or even threatening human lives [44], [45]. For this reason, researchers have characterized DL-specific bugs, *i.e.,* root causes, symptoms, and affected components, considering the peculiarities of DL systems compared to traditional ones, by looking at two different dimensions, *i.e.,* the program level [17], [27], analyzing production code used to train DL models, and the framework level [9], [49], [52], looking at bugs occurring in DL frameworks/libraries used to implement DL systems.

Therefore, ensuring the quality of DL systems is crucial. However, not all problems are necessarily bugs, *i.e.,* not all of them cause a "blocking" system malfunctioning. Developers may occasionally leave behind unresolved issues and acknowledge them, *i.e.,* creating a Self-Admitted Technical Debt (SATD) [34] highlighting the problem through a source code comment or in the issue tracker [21].

There are key differences between bugs and SATD. Bugs represent unintentional errors leading to unexpected or incorrect behavior identified by testing, user reports, and monitoring. When discovered, developers try to fix them, especially for blocking bugs that prevent the system's usage. SATD, instead, refers to the admittance of suboptimal code or design decisions made by developers for expediency, that are not necessarily related to bugs. While, in conventional code, SATD may be related to readability/maintainability or to minor problems that can occur in unlikely usage scenarios [50], in the context of DL systems, the inappropriate selection of a loss function may result in poor model performance.

Characterizing DL-specific SATD is, nevertheless, not as straightforward as it has been done for conventional SATD [4], [21], [34]. DL-based software, similar to ML-based software, includes extra components not typically found in conventional software, which may introduce new domain-specific challenges and, at the same time, may be the origin of new types of TD unseen in conventional software. For instance, ML/DL software performance is highly dependent on the training data quality, which may represent one reason for SATD occurring in those kinds of systems [32]. On the one hand, researchers have

2576-3148/24/$31.00 ©2024 IEEE
DOI 10.1109/ICSME58944.2024.00043

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

characterized SATD in ML applications and tools [6], [7], [32] highlighting that data pre-processing and model generation components are the most prone to TD. On the other hand, Liu *et al.* [23], [24] looked at several DL frameworks to characterize the presence and removal of SATD. However, to the best of our knowledge, their study is focused on the presence of conventional SATD, and they did not look at new types of DL-specific SATD.

This study aims to empirically define a **taxonomy** of DL-specific SATD to design "ad-hoc" strategies to trace, manage, and remove them from DL systems. To achieve the aforementioned goal, we performed a hybrid, cooperative card sorting procedure [41] on a statistically significant sample of 443 SATD, which occurred in 54 Python open-source projects belonging to an initial sample of 100 top-starred Python open-source projects relying on TensorFlow or PyTorch. Such a selection is based on the most popular programming language for ML/DL development and the two most popular DL frameworks, also according to previous software engineering studies [14]. As a result, we obtained a taxonomy of DL-specific SATD, made up of two high-level categories related to infrastructure and DL life-cycle, and featuring a total of seven categories and 41 leaves.

The purpose of designing such a taxonomy is manifold. First, knowing the SATD specific to the DL components/code of a system may aid developers to better understand the scenarios within which it is acceptable to live with a poorly performing system. Furthermore, by comparing DL-specific SATD and bugs, it would be possible to provide a set of guidelines aiding developers in identifying whether or not a "poor" design decision may result in a crash or failure. Finally, by running static code analysis tools on both DL-specific SATD and SATD that can also occur in conventional software, we checked whether DL developers may rely on existing tools to avoid DL SATD.

Replicability: The dataset and scripts of our study are publicly available for replication purposes [33].

Structure of the paper. Section II details the study methodology used to collect SATD from DL systems and to build the taxonomy. The final taxonomy, along with the description of its categories, *e.g.,* through examples, is presented in Section III. Section IV contains a discussion of our findings, with implications for practitioners and researchers, while Section V reviews the threats to the validity. Section VI discusses the related literature, while Section VII concludes the paper and outlines directions for future work.

II. Study Design

The *goal* of the study is to characterize SATD in open-source DL software. The *quality focus* pertains to understanding SATD types specific to DL (and their differences from DL bugs), with the long-term goal of improving the quality of DL systems. The *perspective* is of researchers interested in building and improving approaches to identify and fix DL-specific SATD. The study results can also be beneficial to software developers to better understand the nature of SATD involving DL-specific components. The *context* of the study consists of (i) 443 SATDs sampled from 54 (out of an initial set of 100) open-source DL systems (in the following simply referred to as "projects") hosted on GitHub, and relying on TensorFlow or PyTorch. Based on the stated goal, our study aims to address the following research question:

RQ: ***What types of SATD are found in our studied open-source DL systems?***

This research question aims to qualitatively define a taxonomy of DL-SATD by discriminating SATD as being specific to DL code/components and those that are generic and can also occur in conventional systems. Furthermore, we also investigate the similarities and differences between DL SATD and DL bugs.

A. Data Collection

The initial step focuses on establishing an appropriate selection of projects for analysis. To this aim, we focus on (i) the Python programming language, which is the most popular language being used for DL applications, and (ii) projects using two DL frameworks, *i.e.,* TensorFlow and PyTorch. These are the two most popular DL frameworks, also investigated in previous software engineering work [14] along with Theano [28], which, however, we discarded as no longer active. Also, even when one uses other high-level DL frameworks in Python, such as Keras [12] or HuggingFace transformers [16], TensorFlow or PyTorch are requested.

To identify such projects, we leverage the *Dependents* feature available in the GitHub "Insights", *i.e.,* given a repository, the dependents page shows a list of all other repositories relying on it. Specifically, we downloaded the dependents of the *tensorflow/tensorflow* and *pytorch/pytorch* GitHub projects. As there is currently no GitHub API available for analyzing dependents, this task has been performed by leveraging the *Beautiful Soup* [38] and *requests* Python packages. After removing duplicated instances, we collected 112,952 and 160,365 projects, ordered by their number of stars, depending on the TensorFlow and PyTorch libraries, respectively.

To properly derive a taxonomy of DL-specific SATD, we selected 100 open-source projects from the initial set of candidates. After ordering the projects by stars and excluding forked projects, we manually looked at them to come up with 50 projects relying on TensorFlow and 50 on PyTorch. Our manual analysis aimed to exclude tutorials, code books, toy projects, and projects not primarily written in Python. Furthermore, the number of projects to consider has been determined so that we could have an appropriate number of SATD comments so that we could sample and analyze enough SATD comments for each project.

B. Extracting SATD Comments

Once identified the projects of interest, we proceeded with the extraction of the SATD comments that can be used to elicit our taxonomy. After cloning each project, we restricted our attention to files importing `tensorflow` or `torch`. This

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

was done by analyzing the imports through the Python AST package.

After that, we relied on the *Nirjas* [31] library to extract all source code comments from such files. Specifically, given a Python file as input, Nirjas provides a JSON file containing information about single-line comments and multi-line comments (comment body together with the line number where the comment begins within the file), other than information about source code elements within it. Then, we restricted the set of comments by checking for the presence of an SATD-related keyword. Specifically, we relied on the Keyword-Labeled SATD (KL-SATD) definition provided by Rantala *et al.* [36]. We chose to use a lightweight approach for SATD detection instead of relying on ML detection of SATD, *e.g.,* using the approach by Ren *et al.* [37], since our goal was not to collect all possible SATD comments being admitted, but rather, obtain a statistically significant sample to qualitatively analyze, as well as we wanted to limit the number of false positives. To implement the KL-SATD definition, we relied on regular expressions to identify comments matching one of the following four keywords: `TODO`, `FIXME`, `HACK`, and `XXX` (all matched as case-insensitive whole words). Then, from the comments matching such keywords, we removed those containing nothing but the SATD-related keyword (therefore, not informative).

As a result, we collected a total of 1,306 candidate SATDs across 54 out of 100 initially selected projects. The remaining 46 projects either did not exhibit SATD, or they did not occur in source code files importing TensorFlow or PyTorch. Since manually analyzing all candidate SATD comments in our dataset would be time-consuming, we created a randomly stratified sample, where the strata represent the projects, *i.e.,* SATD comments have been sampled across projects proportionally to the number of candidate SATD comments from the previous step. Given some projects had very few < 5 SATD candidates, we took at least one SATD comment for each project. This resulted in a sample of 443 candidate SATD comments, guaranteeing a margin of error of $\pm 3.71\%$ with a confidence level of 95%. The estimation has been performed relying on the sample size (SS) formula for an unknown population [39]:

$$SS = p \cdot (1 - p) \frac{Z_\alpha^2}{E^2}$$

and SS_{adj} for a known population pop:

$$SS_{adj} = \frac{SS}{1 + \frac{SS-1}{pop}}$$

where p is the estimated probability of the observation event to occur, Z_α is the value of the Z distribution for a given confidence level, and E is the estimated margin of error.

C. Taxonomy Construction

To create a taxonomy of DL-specific SATD, we need to properly identify, among the 443 candidates' SATD, those specific to DL. We consider SATD that may occur in any software system and that were categorized on previous studies on conventional SATD [4], [21], [34] as "not specific".

To this aim, four authors (henceforth referred to as coders), performed a manual classification of the comments, employing a cooperative hybrid card sorting procedure [41], *i.e.,* multiple people formed the taxonomy collectively and iteratively, with no predefined categories. The manual classification was conducted to classify each SATD as (i) *false positive* (*e.g.,* #(todo): $<< name >>$), (ii) *no*: indicating that the SATD is not specific for DL code/components, and (iii) *yes*: the admitted Technical Debt (TD) is specific for DL code/components, and it is unlikely to be annotated in conventional systems. For the latter, the coders provided a label describing the admitted TD type. First of all, we conducted a pilot labeling study where coders, in a plenary meeting, classified 30 SATDs in the dataset to ensure a common understanding of the labeling procedure, and to derive the initial set of categories to use for the classification of the remaining ones. After the pilot was completed, the remaining 413 SATD instances were coded incrementally in five rounds involving 50, 100, 100, 100, and 63 SATDs, respectively.

Within each round, two coders independently validate the SATD assigning one of the previously defined categories or else adding a new one, when needed, using an online spreadsheet displaying the categories defined so far by the coders. The latter has been done to ensure consistent naming without introducing substantial bias. At the end of each round, the remaining two coders checked all the instances considered relevant by at least one of the original coders. It is important to note that we could not estimate the reliability of the study using inter-rater agreement due to the incremental process used to define the DL-specific SATD categories. However, to limit agreement by chance, the two coders not involved in the labeling process discussed and also checked instances where there was an agreement.

Concerning the saturation, on the one side, we started drafting our taxonomy at the end of the third round. Then, we performed the subsequent rounds to check the extent to which saturation was reached. The fourth round introduced only two leaves to the taxonomy, *i.e.,* "Inference→Inappropriate Output Postprocessing" and "Training Logic→Numerical Bug". The fifth round did not introduce any new category.

At the end of the third round, the four coders worked together to derive a first version of the taxonomy consisting of 37 DL-specific SATD, organized into six categories. The initial taxonomy was revised and refined at the end of the fifth round, ending with a taxonomy consisting of 41 DL-SATD types organized into seven categories, further grouped into two high-level categories.

III. Study Results

As a result of our card sorting procedure, we defined a taxonomy of DL-specific SATD, depicted in Fig. 1. On each leaf, the figure also reports the number of occurrences from the sample classified into a given category. The upper-level nodes report the sum of their leaves.

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

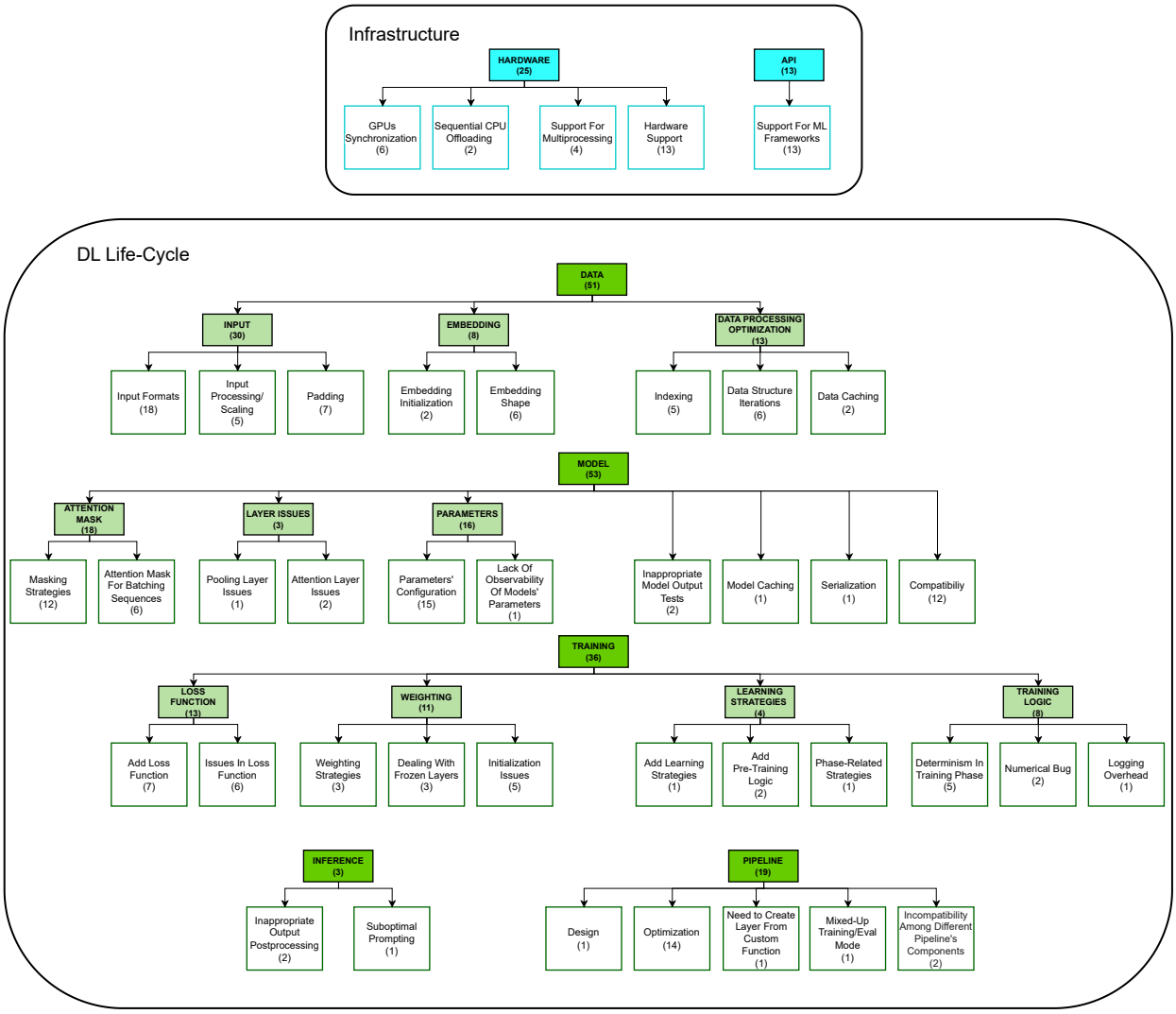

Fig. 1: Taxonomy of DL-specific SATD

As the figure shows, the taxonomy is organized into two high-level categories, related to Infrastructure and DL Life-Cycle, further specialized into seven categories and 41 leaves.

In the following, we discuss the taxonomy along the different categories, and report, when useful, some examples. For such examples, we provide traceability in Table I (the full sample traceability is in our replication package [33]). Last but not least, for each category, we summarize the main findings and their implications.

A. Infrastructure

SATD related to infrastructure pertains to two categories: one related to the usage of hardware (*e.g.,* GPUs), and another one to the interaction with external APIs for developing the DL framework/application.

Hardware. This category accounts for SATD dealing with hardware components, *e.g.,* GPUs or TPUs, used for training the DL model or during inference once the trained model is deployed in a production environment. In ALPA-PROJECTS/ALPA, during the process of integrating PyTorch, a developer commented on the need to *"add support for CUDA input tensor"* (E_1). The application was not designed to cope with GPUs, while CPUs and meta-input tensors were already handled. Furthermore, in the presence of multiple devices, developers may face challenges regarding the parallelization of certain operations and their synchronization. As an example, in TENSORFLOW/LINGVO, a developer asks for the possibility of implementing SPMD (Single Program, Multiple Data) having multiple units cooperating in the execution of a program to obtain results faster, *i.e., "Configure for spmd"* (E_2). When developing DL applications, the speed for training

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I: List of SATD examples (to open, add `https://github.com/` before)

ID	Category	URL
		HARDWARE
E1	Hardware Support	alpa-projects/alpa/blob/824f2ff/alpa/torch/tensor_utils.py#L74
E2	Support For Multiprocessing	tensorflow/lingvo/blob/cd0a505/lingvo/jax/layers/repeats.py#L179
E3	Support For Multiprocessing	huggingface/diffusers/blob/ba59e92f/src/diffusers/pipelines/unidiffuser/pipeline_unidiffuser.py#L106
		API
E4	Support For ML Frameworks	ivy-llc/ivy/blob/975c8fc/ivy/functional/frontends/tensorflow/general_functions.py#L483
E5	Support For ML Frameworks	microsoft/muzic/blob/5b3890d/emogen/linear_decoder/command_seq_generator.py#L438
		DATA
E6	Input: Input Processing/Scaling	IntelLabs/nlp-architect/blob/88b3236/nlp_architect/models/tagging.py#L276
E7	Input: Input Processing/Scaling	huggingface/diffusers/blob/ba59e92f/examples/community/pipeline_zero1to3.py#L667
E8	Embedding: Embedding Initialization	huggingface/transformers/blob/5936c8c/src/transformers/models/encodec/convert_encodec_checkpoint_to_pytorch.py#L230
E9	Embedding: Embedding Initialization	microsoft/LMOps/blob/909022d/minillm/transformers/src/transformers/models/opt_parallel/modeling_opt_parallel.py#L856
E10	Data Processing Optimization: Data Caching	AUTOMATIC1111/stable-diffusion-webui/blob/5ef669d/extensions-builtin/LDSR/sd_hijack_ddpm_v1.py#L600
E11	Data Processing Optimization: Indexing	openGVLab/InternGPT/blob/2aceb06/iGPT/models/grit_src/third_party/CenterNet2/detectron2/modeling/proposal_generator/rrpn.py#L184
E12	Data Processing Optimization: Data Structure Iterations	huggingface/diffusers/blob/ba59e92f/scripts/convert_kandinsky_to_diffusers.py#L837
		MODEL
E3	Attention Mask: Masking Strategies	flairNLP/flair/blob/603de99/flair/models/lemmatizer_model.py#L324
E14	Attention Mask: Attention Mask For Batching Sequences	alpa-projects/alpa/blob/824f2ff/examples/llm_serving/model/wrapper.py#L575
E15	Layer Issues: Pooling Layer Issues	apache/tvm/blob/c318fa8/tests/python/frontend/pytorch/test_forward.py#L2533
E16	Parameters: Parameters' Configuration	alpa-projects/alpa/blob/824f2ff/alpa/torch/ops/mapping.py#L121
E17	Parameters: Lack of Observability of Models' Parameters	EleutherAI/gpt-neo/blob/23485e3/utils.py#L178
E18	Inappropriate Model Output Tests	Sygil-Dev/sygil-webui/blob/91a4eba/ldm/models/diffusion/ddpm.py#L1249
E19	Model Caching	Sygil-Dev/sygil-webui/blob/91a4eba/scripts/scn2img.py#L147
E20	Serialization	keras-team/keras/blob/eb12517/keras/legacy/saving/json_utils.py#L114
E21	Compatibility	FedML-AI/FedML/blob/e94d88/python/fedml/core/distributed/communication/s3/remote_storage.py#L317
E22	Compatibility	microsoft/LMOps/blob/909022d/structured_prompting/fairseq-version/fairseq/examples/roberta/wsc/wsc_task.py#L56
		TRAINING
E23	Loss Function: Issues in Loss Function	microsoft/LMOps/blob/909022d/understand_icl/fairseq/examples/MMPT/mmpt/losses/nce.py#L19
E24	Loss Function: Add Loss Function	huggingface/transformers/blob/5936c8c/src/transformers/models/longt5/modeling_longt5.py#L2076
E25	Weighting: Weighting Strategies	tensorflow/lingvo/blob/cd0a505/lingvo/jax/train.py#L469
E26	Weighting: Dealing With Frozen Layers	huggingface/transformers/blob/5936c8c/src/transformers/models/sam/modeling_tf_sam.py#L577
E27	Learning Strategies: Phase-Related Strategies	microsoft/LMOps/blob/909022d/understand_icl/fairseq/examples/MMPT/mmpt/evaluators/predictor.py#L541
E28	Training Logic: Determinism in Training Phase	openai/shap-e/blob/0b26c72/shap_e/models/nn/ops.py#L407
E29	Training Logic: Numerical Bug	huggingface/transformers/blob/5936c8c/src/transformers/models/speech_to_text/modeling_tf_speech_to_text.py#L1292
E30	Training Logic: Logging Overhead	Stability-AI/stablediffusion/blob/cf1d67a/ldm/models/diffusion/ddpm.py#L1391
		INFERENCE
E31	Inappropriate Output PostProcessing	openGVLab/InternGPT/blob/2aceb06/iGPT/models/grit_src/third_party/CenterNet2/tools/deploy/export_model.py#L98
E32	Suboptimal Prompting	oobabooga/text-generation-webui/blob/a0d99dc/extensions/openai/completions.py#L193
		PIPELINE
E33	Design	microsoft/LMOps/blob/909022d/minillm/transformers/examples/research_projects/wav2vec2/run_asr.py#L193
E34	Need to Create Layer from Custom Function	IntelLabs/nlp-architect/blob/88b3236/nlp_architect/nn/torch/quantization.py#L65
E35	Mixed-Up Training/Eval Mode	allenai/allennlp/blob/80fb606/allennlp/interpret/influence_interpreters/simple_influence.py#L164
E36	Incompatibility Among Different Pipeline's Components	huggingface/transformers/blob/5936c8c/src/transformers/configuration_utils.py#L341

and inference is not always the only performance requirement to look at. Indeed, we found cases where developers must consider the balance between memory usage and speed. As an example, this involves relying on sequential CPU offloading. Indeed, while the latter saves memory consumption, it also slows down the overall inference process since sub-modules are moved to GPU as needed while returned to the CPU as soon as a new module runs, *i.e., "support for moving submodules for components with enable_model_cpu_offload"* in HUGGINGFACE/DIFFUSERS (E_3).

Finding 1 (Hardware): One of the main components developers need to care about is *design for change* for what concerns the infrastructure, to easily support different hardware, as well as parallelization and other optimization elements. Also, it might be valuable to give developers automated hints about possible performance smells that may occur in DL-specific code.

API. This category accounts for SATD due to the usage and integration of DL frameworks. Most of the problems are related to features that DL frameworks do not support. As an example, in UNIFYAI/IVY, there exists a need to use negative indexing. Unfortunately, the used DL framework does not accommodate this requirement, *i.e., "#ToDo: find a way around for negative indexing, which torch does not support"* (E_4). In some cases, the lack of support for a feature in a DL framework is circumvented through a workaround. This is the case of MICROSOFT/MUZIC, where a developer acknowledges the presence of a workaround to be removed as soon as the DL framework will explicitly deal with that specific condition, *i.e., "#TODO replace 'nonzero(as_tuple=False)' after TorchScript supports it"* (E_5).

Finding 2 (API): API-related SATD is usually related to unavailable features, and it may require a temporary workaround. To this extent, features to aid developers to automatically open issues to suggest the desired features will be worthwhile, but also specialized code recommenders suggesting possible workarounds.

B. DL Life-Cycle

This high-level category embraces SATD related to various aspects of the DL development life-cycle. Specifically, it features TD related to (i) the type of data and how it is shaped for training purposes, (ii) the training process, such as weighting strategies and loss functions, (iii) the inference process in terms of prompting and post-processing of the produced outcome, (iv) the structure and configuration of the trained DL models, and, (v) the DL pipeline as-a-whole.

Data. This category groups SATD related to the format/shape of the data used to train a model. This may concern the *Input* format itself and the extent to which such an input has been properly preprocessed. The latter may nega-

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

tively impact the model's convergence during training, *e.g., "#TODO: implement method to get only valid logits from the model itself"* in INTELLABS/NLP-ARCHITECT (E_6), or else its performance, *e.g., "# todo in original zero123's inference gradio_new.py, model.encode_first_stage() is not scaled"* in HUGGINFACE/DIFFUSERS (E_7).

Other common concerns are related to the *Embeddings*, *i.e.,* input representation capturing meaningful relationships between different input terms. SATD may concern embedding initialization, *e.g.,* in HUGGINGFACE/TRANSFORMERS, a developer introduced a workaround to handle the type of embedding to use during training as it mismatched with the model, *i.e., "#HACK otherwise .embed gets initialized with .embed_avg too"* (E_8). Other problems are related to their shape, as in MICROSOFT/LMOPS, where a developer postponed a proper embedding shape consistency check, *i.e., "# TODO: check tied"* (E_9).

Another aspect that is worth mentioning concerns how inappropriate input data management can lead to efficiency problems in the model training phase, hence requiring a *Data Processing Optimization*. As an example, developers felt it appropriate to reduce—through caching—the overhead associated with data loading during the training process, specifically in the presence of large datasets or complex data pre-processing, *e.g., "todo load once not every time"* in AUTOMATIC1111/STABLE-DIFFUSION-WEBUI (E_{10}). Problems related to indexing ignored data may lead to performance issues too, *e.g.,* in OPENGVLAB/INTERNGPT, a developer acknowledged the poor performance resulting from unneeded operations due to the inappropriate handling of the input data, *i.e., "# TODO wasted indexing computation for ignored boxes"* (E_{11}).

Last but not least, we also found concerns related to the need for better data padding (*e.g.,* add support for padding in multiple directions), or to improve data structure iterations, mainly to improve the overall performance, *e.g., "#TODO maybe document and/or can do more efficiently (build indices in for loop and extract once for each split?)"* in HUGGINFACE/DIFFUSERS (E_{12}).

Finding 3 (Data): Developers may need suitable support to check the conformance of input data with the model and, if needed, pre-process or adapt them. Moreover, the way data is read/processed can introduce performance antipatterns, leaving room for optimization.

Model. This category includes SATD concerning the design and setting of the DL model. It is worth mentioning problems faced when defining the DL model architecture in terms of attention masks, layers, and other relevant parameters.

Attention Masks are binary masks determining which tokens should be attended to or ignored. SATD we found relate to the choice of masking strategies, or to determining attention masks for batch sequences. In the FLAIRNLP/FLAIR project, we found a comment highlighting the need to check the attention masking process, ensuring that certain elements are ignored, *i.e., "mask out vectors that correspond to a dummy symbol (TODO: check attention masking)"* (E_{13}). Other attention mask-related SATD concerns the masking for batch sequences, *e.g.,* in ALPA-PROJECTS/ALPA a SATD mentions *"#Skip the check in DistributedPhysicalDeviceMesh::shard_args for attention cache. We need this hack because attention_cache is a batch var but alpa doesn't implement a fast path for batch vars."* (E_{14}).

SATD related to *Layers* concerns the presence, in the model, of specific types of layers. We found SATD related to the need to add attention layers or pooling layers. Such pooling layers are typically used (*e.g.,* in Convolutional Neural Networks) to reduce input sizes among different layers. In the APACHE/TVM project, we found a SATD related to the need for fixing a potential issue that could occur in certain circumstances because of the lack of proper pooling: *"#TODO: Fix VGG and AlexNet issues (probably due to pooling)"* (E_{15}).

Once a DL model architecture has been defined in terms of layers and their interconnection, a major issue is the inappropriate configuration of the model *Parameters*. In some cases, such as in ALPA-PROJECTS/ALPA, this is likely done to improve both flexibility and re-usability of the code, *i.e., "…parameterize this! don't assume NCHW format."* (E_{16}). In other cases, the code does not allow to inspect all model parameters, leading to observability and maintainability issues. As an example, in ELEUTHERAI/GPT-NEO, a SATD mentions *"TODO: how to get un-trainable dim-names too, batch etc."* (E_{17}).

Other model-related SATD concerns (i) inappropriate model output tests, (ii) model caching, (iii) enhancing serialization features, and (iv) compatibility between different DL components/models. Inappropriate model output test concerns the need to handle cases where the model can produce multiple outputs (*e.g.,* multiple solutions ranked using beam search). As an example, in SYGIL-DEV/SYGIL-WEBUI, a developer wants to ensure multiple outputs are never produced, otherwise the output shape itself changes, and this may create issues, *i.e., "#todo cant deal with multiple model outputs check this never happens"* (E_{18}).

Model caching simply concerns ensuring a pre-trained model is loaded from a local cache and not downloaded (to save time), *e.g.,* in SYGIL-DEV/SYGIL-WEBUI a SATD mentions *"#todo: load model from pre-trained keras into user .cache folder like transformers lib is doing it."* (E_{19}).

Serialization concerns the program's features to store trained models on files. We found SATD related to enhance this, and, specifically, in KERAS-TEAM/KERAS, to give the possibility to define saved parameters for different scopes, a developer states: *"#TODO(…): Add TF SavedModel scope."* (E_{20}). Such an option, as also discussed in a Stack Overflow post[1], would be very useful in the context of transfer learning,

[1]https://stackoverflow.com/questions/55245813/saving-restoring-weights-under-different-variable-scopes-in-tensorflow

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

to better save models for multiple fine-tuning conducted for different purposes.

Finally, developers highlight problems in guaranteeing the compatibility among different DL components/models, such as the integration of different DL frameworks, *e.g., "#TODO: added torch model to align the Tensorflow parameters from browser"* in FEDML-AI/FEDML (E_{21}), or the need to rely on workarounds to properly integrate pre-trained models due to different choice for handling specific input data and/or parameters, *e.g., "#hack to handle GPT-2 BPE, which includes leading spaces"* in MICROSOFT/LMOPS (E_{22}).

Finding 4 (Model): The problem of model tuning and, in general, of model configuration is a long-standing one for which reusing previously-adopted solutions may not necessarily be the best option. As far as pre-trained models are concerned, challenges developers face in their integration may be mitigated by providing examples of use, as many models hosted on HuggingFace do [5].

Training. This category considers SATD dealing with the training process in terms of selecting an appropriate loss function, initializing the model parameters, such as weighting strategies, together with adequate learning strategies, or a sub-optimal implementation of the whole training logic.

As regards the usage of the appropriate *Loss Function*, we encountered two different classes of problems. On the one hand, someone may use loss functions without properly adjusting their parameters, *i.e.,* keeping their default values, such as *"#TODO: define temperature."* in MICROSOFT/LMOPS (E_{23}). The temperature is a parameter used in DL models to determine the smoothness of the (softmax) output distribution. That is, a low temperature means more deterministic outputs, whereas a high temperature means more variation. This is also a very important parameter at inference time to make the model more coherent with its training data or more "creative" (as it has been studied in recent work [13]). On the other hand, there are cases requiring the implementation of a customized loss function since it has been proven to be more suitable for the scenario at hand. As an example, in HUGGINGFACE/TRANSFORMERS, a developer acknowledges the need to *"Add z_loss"* (E_{24}), linking this requirement to an external project which, in a similar scenario, is following the same strategy.

Adopting an appropriate *Weighting* strategy is beneficial for several reasons, such as the appropriate handling of the class imbalance problem or the improvement in terms of model generalizability, *i.e.,* guaranteeing good performance on unseen data. As an example, in TENSOFLOW/LINGVO, a developer admitted the need to obtain the initial random seeds used for initializing the model's parameters directly from the model definition, rather than manually specifying them, *i.e., "#TODO: Retrieve seeds from model definition instead"* (E_{25}), likely to ensure consistency and reproducibility. Other weighting problems are related to frozen layers. In some cases, developers are wondering why the weights of a frozen layer receive a (useless) random initialization, like in HUGGINGFACE/TRANSFORMERS: *"TODO ... Why is a non-trainable weight randomly initialized?"* (E_{26}). In other cases, developers suggest explicitly freezing a layer to prevent its retraining or fine-tuning, hence avoiding a possible "forgetting" problem, *i.e.,* loosing knowledge previously acquired. Finally, weighting problems concern initialization issues, *i.e.,* problems occurring when initializing weights from certain (*e.g.,* incompatible) sources.

For what concern the *Learning Strategies*, SATD is related to (i) improving learning-related features, such as adding new learners, (ii) adding specific (customized) logic for using pre-trained models (and eventually fine-tuning them), or (iii) adding specific logic for certain phases of the training (*i.e.,* either for the forward or the backward phase), *e.g.,* in MICROSOFT/LMOPS a SATD mentions *"#TODO special forwarding logic here."* (E_{27}).

For what concerns the *Training Logic*, it is worth mentioning the need to guarantee determinism within the training process that would ensure both consistency and reproducibility for the trained model, *e.g., "#TODO shuffle deterministically when not self.training"* in OPENAI/SHAP-E (E_{28}). Indeed, DL frameworks provide explicit APIs to initialize the random seed. For example, TensorFlow uses the `enable_op_determinism` API or the `TF_DETERMINISTIC_OPS` environment variable, while Keras uses the `set_random_seed` API to initialize the random seed. While a varying random seed may allow for re-shuffling the starting point over multiple experiments, a constant one ensures the reproducibility of the training process. Other problems related to the training logic concern numerical bugs, *e.g.,* due to repeated approximations, like in HUGGINGFACE/TRANSFORMERS: *"#TODO (...): investigate why Speech2Text has numerical issues in XLA generate."* (E_{29}).

Finally, we found SATD related to possible overhead introduced by (excessive) logging, like in STABILITY-AI/STABLEDIFFUSION where a developer suggests to disable some expensive logs: *"#TODO: maybe disable if too expensive"* (E_{30}). Clearly, TD like this one introduces trade-off decisions, *i.e.,* pondering between performance (with limited logging) and training process troubleshooting.

Finding 5 (Training): Like model calibration (which requires complex choices), the lack of setting a proper training phase may lead to performance problems or sub-optimal results. Sometimes developers, possibly because of the lack of effort to properly customize the training or to experiment with different options, decide to live with that.

Inference. This category includes SATD related to inappropriate post-processing of the outcome generated when running the trained model, as well as using suboptimal prompting. For

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

the former, in the OPENGVLAB/INTERNGPT project, there is a comment highlighting the lack of a function to integrate the output of the inference step with subsequent processing or actions, *i.e., "...inference is now missing post[-]processing glue code"* (E_{31}). For what concerns prompt-related SATD, one example occurred in the OOBABOOGA/TEXT-GENERATION-WEBUI project, reporting the need to improve the prompting for inference purposes, *i.e., "#XXX User: prompt here also"* (E_{32}). Determining the proper prompt to be used during inference—*i.e.,* the input query to be seeded into the model, possibly indicating information such as the user's role, the task context, and required output format—has already been studied in previous work [47]. Last but not least, it must be noted that one problem that may affect the inference phase is the setting of the temperature (as we discussed for the training phase) or other hyper-parameters that affect the inference phase (*e.g.,* the beam search, or, for some text transformer models, the parameter to penalize repetitions). However, we did not find such instances in our sample.

Finding 6 (Inference): Unsurprisingly, developers highlight the need for improvement, especially for what concerns prompt engineering. While there is intense work in this area, development tools could be enhanced to provide recommendations for suitable prompts for different types of inference.

Pipeline. This category groups SATD dealing with the setting of the DL pipeline in terms of its design and optimization.

First of all, we found design issues affecting the usage of DL-pipelines, *e.g.,* the need to handle certain features through the pipeline, and consequently, for a function, to return a pipeline object, as in MICROSOFT/LMOPS (E_{33}): *"#TODO(...) return a pipeline (e.g., from jiwer) instead? Or rely on branch predictor as is."*

Several projects in our dataset faced problems in terms of deciding what has to be moved outside the pipeline, *e.g., "Prepare extra step kwargs. TODO: Logic should ideally just be moved out of the pipeline"* likely to improve modularity and maintainability of the source code. In the examples we found, this was mainly related to extra processing steps, that need to be properly factored out as functions, and not in-lined directly as code in the pipeline. In other circumstances, a suggested (and deferred) change concerns creating a predefined layer to be used in the pipeline, as in INTELLABS/NLP-ARCHITECT: *"#TODO(...) future work, implement a layer that uses this function that gives a more comfortable."* (E_{34}).

A recurring concern in pipelines is to force the models' mode, *e.g., "#TODO (...): should we make sure 'model' is in "train" or "eval" mode here?"* in ALLENAI/ALLENNLP (E_{35}). In eval mode, a model works differently, *e.g.,* it does not use dropout nodes. While the current (default) mode in the code could be the desired one, an explicit set may avoid possible future bugs.

Finally, some pipeline-related SATD concerns the compatibility across several components. As an example, in the HUGGINGFACE/TRANSFORMERS project, we found a comment admitting the need to align the configurations between the tokenizer and the models used in the code, *i.e., "...TODO: eventually tokenizer and models should share the same config"* (E_{36}), likely to improve both consistency and compatibility between the tokenizer and the models.

Finding 7 (Pipeline): On the one hand, developers may need better support (*e.g.,* low-code tools or IDE-integrated recommenders) to ease the composition of different DL pipeline blocks. On the other hand, suboptimal decisions on what should or should not be in the pipeline may lead to the definition of DL pipeline smells and related refactoring actions.

IV. DISCUSSION

In the following, we first discuss, based on the results presented in Section III, the emerging differences between DL bugs—studied in previous research [9], [15], [17], [43], [52]—and DL-SATD, and the adequacy of existing static analysis tools to identify DL-SATD. Then, we outline implications for different stakeholders.

A. Differences between DL SATD and DL bugs

Our qualitative findings stress the importance of ensuring the quality of DL systems, not only in terms of avoidance of system crashes, mainly due to the presence of "blocking" DL bugs, but also in terms of awareness of SATD, describing the presence of unresolved issues that have to be fixed later on in the system.

At first glance, our SATD taxonomy would appear (by many category names) similar to some DL bug taxonomies, among others the one by Humbatova *et al.* [17], *i.e.,* the leaves have similar naming but different meanings in terms of the type of effect on the system, *e.g.,* on the one side, a system fails to accomplish its task, or it crashes, while on the other side, there may be poor performance in terms of training time or convergence. A detailed mapping between our taxonomy and the one by Humbatova *et al.* is in our replication package [33].

One key difference between our study and what Humbatova *et al.* [17] did is that we analyzed source code comments (SATD) left by developers during their coding tasks, while they focused on issues, bug fix commits, Stack Overflow discussions, and interviews with practitioners. Issues preventing the system from properly working are unlikely to be admitted in source code comments. Instead, typically one (i) submits an issue, or (ii) asks for help on Q&A forums by reporting the failing code snippets and describing what is the desired behavior. As an example, problems dealing with the wrong reference to the GPU device[2] make the DL-system fail. However, developers might decide to leave with performance

[2]https://stackoverflow.com/questions/40726039/tensorflow-cuda-visible-devices-doesnt-seem-to-work

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

issues due to a lack of parallelization of certain operations and their synchronization.

Furthermore, in DL systems, one possible root cause for both bugs and SATD deals with tensor types and their shapes. While mismatches between tensors' shape/type represent a blocking issue, DL-SATD mainly describes problems negatively impacting model convergence, as well as the overhead of data loading during training.

Having said that, there are still commonalities among DL SATD and DL bugs in terms of (i) negative impact on software quality, (ii) contributing to software maintenance challenges, and (iii) need to be properly documented even if within different sources, *i.e.,* annotations in code and issues.

B. Adequacy of state-of-the-practice static analysis tools

Before thinking about DL-specific SATD recommenders and repayment tools, it would be worthwhile to check whether existing state-of-the-art static analysis tools can at least help spot possible DL-specific TD.

Therefore, we investigated whether code snippets affected by DL-specific SATD in our sample contain at least one warning raised by *Prospector* [20]. Prospector is an aggregator of several static analysis tools for Python, including PyLint [25], Flake8 [42], or the MyPy type checker [30]. We conjectured that conventional SATD, such as *"// this method is a nightmare"* or *"TODO: refactor this method is too long"*, can be easily identified by Prospector, while comments such as *"#TODO: Enable multi-device support"* or *"#TODO(...) - attention mask is not used"* are unlikely to be detected by static code analysis tools. To verify our conjecture, we ran Prospector on the Python files containing SATDs in our sample and manually checked its reports to verify whether or not among the issues being raised there exist some that can be used as a proxy to detect the SATD.

Our manual analysis (detailed results can be found in our replication package [33]) reveals that out of 432 SATDs in our sample, only 21 (of which only 4 are DL-specific) are somehow related to a warning issued by Prospector. For instance, the conventional SATD *"#todo no isp_model?* is identifiable by the violation **Unused argument 'isp_model'**, or else *"#TODO (...): The current implementation is ugly. Refactor..."* can be associated with the **Too many statements** violation. Looking into DL-specific SATD, we found only two SATD instances somehow related to a Prospector warning. The first one, (repeated in three code snippets), *"#TODO: don't match quantizer.weight_proj"* matched a type/signature violation (SIGNATURE INCOMPATIBLE WITH SUPERTYPE), while *"#TODO: maybe have a cleaner way to cast the input (from 'ImageProcessor' side?)"* relates to a type violation (CANNOT ASSIGN TO A TYPE.)

In conclusion, the results of this investigation indicate that currently-used static analysis tools cannot capture the essence of DL-specific TD, and, rarely help to identify TD in general (for the latter, confirming previous findings by Rantala *et al.* [35]). This enforces the need for specialized approaches to properly detect and track them during software evolution.

C. Implications

Educators should illustrate how to favor design for change in DL-intensive systems. As we found, this may concern, on the one hand, easing the adaptation to different hardware infrastructures, and, on the other hand, making DL pipelines flexible for what concerns the addition/change of their stages.

Furthermore, although students and professionals today can quickly become proficient with new technologies—thanks to the wide amount of formal and informal documentation available and to advanced tools, including those based on Large Language Models, that support software development—there is still a lack of effective guidance for recognizing, understanding, and correcting poor design and coding practices.

As far as **Practitioners** are concerned, a key lesson is related to deciding when a problem in a DL-intensive system is a bug and therefore needs to be fixed as soon as possible, or when it is a SATD and, the system can be released by leaving with that. To this extent, developers should employ a suitable evaluation to identify significant (blocking) deviations from the expected behavior from sub-optimal model accuracy or performance problems whose fix can be postponed. Moreover, as it also happens for conventional systems, developers should be prone to admit TD, as previous work suggested that this may not always be the case in the industry [50].

A further implication for practitioners, which could also inadvertently affect other stakeholders, is that, similar to certain types of conventional SATD, DL-specific TD may imply trade-off decisions. As an example, just as performance and maintainability can often be conflicting goals in conventional systems, a faster training process in DL systems may come into conflict with other objectives, such as the ability to effectively monitor the training process (*e.g.,* through logs) and, ultimately, the effectiveness of the trained models.

Similarly to what has been done for conventional systems [37], **Researchers** should develop approaches to (i) identify the presence of "DL-specific" bad smells in the source code, and recommend refactoring actions, or else the admittance of a TD, or (ii) propose solutions to repay DL-specific TD.

V. THREATS TO VALIDITY

Construct validity threats concern the relationship between theory and observation. A possible threat is related to the subjectiveness of our analysis, as the produced taxonomy is based on our interpretation of the SATD comments. We mitigated this threat by having two coders who inspected each SATD candidate, and two more coders who checked all the previously-coded categorizations and resolved the controversial cases.

Internal validity concerns factors, internal to our study, that could have influenced the results. A threat is related to the identification of SATD candidates. The keyword-based method employed [36] ensures high accuracy, yet it might bring some selection in the sample, *e.g.,* by excluding certain types of SATD because they are not tagged with given keywords. At the same time, given the completely different domain

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

than conventional SATD, an ML-based detection might have introduced more noise in the analysis and still not ensured enough coverage.

Conclusion validity threats mainly concern the representativeness of the sample considered for the analysis. On the one hand, we have sampled a statistically significant number of SATD candidates. On the other hand, such a sample has been extracted on a fairly limited number of projects.

External validity threats concern the generalizability of our findings. There are three generalizability limitations. First, the study has been limited to Python and the dependents of TensorFlow and PyTorch. While this decision has been justified in Section II given the popularity of Python and both DL frameworks, it is possible that projects developed with other pieces of technology may exhibit different forms of SATD. Second, given the qualitative nature of the study, the analysis has been conducted on a relatively limited number of projects (54), and therefore the conclusions may not generalize beyond such projects. Third, it might be worthwhile to study DL SATD in the closed-source, by employing different empirical methods, *e.g.,* surveys or interviews.

VI. Related Work

Considering the scope of our work, we focus on three distinct aspects: (i) TD and SATD in conventional software, (ii) studies on bugs in ML and DL software, and (iii) SATD in ML software.

A. Conventional TD and SATD

In recent years, the research community has extensively explored Technical Debt as a communication medium among developers and managers to address development issues [1], [8], [19], [40]. Lim *et al.* [22] emphasized the intentional introduction of TD, while Ernst *et al.* [11] and Zazworka *et al.* [51] stressed the importance of TD awareness for effective management and mitigation of its negative impact on software quality.

In their seminal work, Potdar and Shihab [34] found that developers tend to "self-admit" TD (SATD) through source code comments. Based on this finding, da Silva Maldonado and Shihab [26] properly categorized different types of SATD, *i.e.,* defect, design, documentation, requirement, and test debts, further refined by Bavota and Russo [4]. Other work looked at commonalities and differences between SATD within open-source and industry [50], *e.g.,* industrial developers are less prone to admit TD because of organizational guidelines and career concerns.

Source code comments are not the only means developers use to trace the presence of TD in the developed code. Xavier *et al.* [48] studied SATD in issue trackers, showing that only a small portion of them (29%) can be traced to source code.

Researchers also put effort into properly detecting and categorizing SATD. For instance, Rantala *et al.* [36] analyzed SATD comments with a detector for Keyword-Labeled SATD, while Ren *et al.* [37] proposed an approach relying on a Convolutional Neural Network to classify source code comments as SATD or non-SATD, as well as to identify key phrases and patterns in code comments that are most relevant to SATD.

In our work, we focus on SATD categories that are different, and complementary to conventional SATD. As we showed, even SATD related to the need for "design for change" pertains to specific changes concerning model deployment on hardware or DL pipeline refactoring.

B. Studies of bugs in ML and DL Software

Different authors studied bugs in ML and DL frameworks and applications. Zhang *et al.* [52] conducted an empirical study to examine the root causes and symptoms of 175 TensorFlow bugs, as well as how such bugs have been detected and located, highlighting five common strategies for bug detection and localization. Jia *et al.* [18] looked deeper at TensorFlow bugs, analyzing their symptoms and causes, distribution across various components of the library, and repair models. Their findings emphasized the significance of causes over symptoms, as well as commonalities between bugs in traditional software and those in TensorFlow. On the same line, Yang *et al.* [49] studied bugs in DL frameworks at the source code level. They sampled 1,127 bugs from eight DL frameworks, and manually labeled them according to bug type, root causes, and symptoms.

Humbatova *et al.* [17] developed a taxonomy of faults in DL systems by manually analyzing 1,059 open-source artifacts and validated it through a survey. The obtained taxonomy featured five primary categories and 375 instances of 92 distinct failure types.

Tambon *et al.* [43] conducted an empirical study investigating silent bugs in TensorFlow, categorizing them into 7 scenarios and 4 impact levels based on their effects on DL systems. A survey with 103 TensorFlow users underscored the significant impact of silent bugs in DL frameworks, largely attributed to the stochastic nature of such systems.

Ho *et al.* [15] analyzed 194 bugs from the PyTorch framework, investigating their root causes, symptoms, and repair patterns. The findings have been compared with bugs occurring in TensorFlow [18], highlighting both similarities regarding the bug's root causes, symptoms, and repair patterns, and differences in terms of which components are prone to bugs.

Going deeper into bugs occurring in ML-intensive systems, Morovati *et al.* [29] manually examined 386 issues from ML-based systems employing TensorFlow, PyTorch, and Keras. The findings revealed that ML components exhibit higher error rates compared to non-ML ones, as well as ML-related bugs require more effort to be fixed than conventional ones.

All the aforementioned pieces of research studied bugs in DL frameworks and/or systems. As we have shown in Section III, and discussed more in detail in Section IV-A, one key difference between DL bugs and DL (self-admitted) TD is that the former may cause crashes or functional errors in common system usages, whereas the latter concern sub-optimal solutions, maintainability problems or, at most, errors occurring in corner-case circumstances.

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

C. SATD in ML Software

The most related work to ours is by Obrien *et al.* [32] which analyzes SATD occurring in ML-based software, proposing a classification scheme accounting for both conventional and ML-specific SATD. While their scope is almost similar to ours, their dataset only accounts for (shallow) ML-based applications while we considered the most forked projects relying on TensorFlow or PyTorch. Furthermore, while our taxonomy has been constructed by considering as dimensions infrastructure and DL lifecycle, including also SATD dealing with inappropriate configuration and setting of the hardware devices to be used for training and inference, their taxonomy considers as dimensions quality aspects such as awareness, modularity, readability, and performance. Last, but not least, there are only 8 overlapping categories across the two taxonomies—the ones related to configurable options and data dependency—highlighting the complementarity between them.

Bhatia *et al.* [6] investigated the presence of SATD in 318 open-source ML projects across five domains, and compared them with 318 non-ML projects, in terms of their nature and, also, analyzed their survival over time. Their findings revealed that SATD in ML projects mostly pertains to data pre-processing and model generation.

Liu *et al.* [23] investigated SATD in the seven most popular open-source DL frameworks. They found a high prevalence of SATD in all studied frameworks, with design debts being the most common. In follow-up work, Liu *et al.* [24] studied the removal of SATD in the same DL frameworks. They found that design debts were most reported and removed quickly, while documentation and defect debts were least self-removed. While the studies by Liu *et al.* [23], [24] focused on conventional SATD, we elicited a taxonomy of DL-specific SATD.

Finally, Bogner *et al.* [7] conducted a systematic mapping study to characterize TD and antipatterns in AI-based systems. Their study led to the identification of four broad categories of AI TD (data, model, configuration, and ethical debt). Additionally, they created a catalog of 72 antipatterns, primarily related to data and model, and identified 46 solutions to reduce the accumulation of TD. Our study has a different setting—analysis of code comments instead of mapping study—relying on actual SATD instances. For this reason, we were able to identify concrete instances of SATD and low-level, code-related problems.

VII. Conclusion and Future Work

The implementation of Deep Learning (DL)-intensive software requires coping with several challenges, including the proper usage of software and hardware infrastructure, as well as the proper creation, tuning, training, and usage of DL models. Sometimes, such challenges may lead to bugs. In other cases, developers decide to leave with sub-optimal solutions, introducing DL-specific Technical Debt (TD).

This paper qualitatively investigates DL-specific Self-Admitted Technical Debt (SATD), *i.e.,* SATD occurring in DL-intensive systems, and not related to conventional problems such as maintainability, understandability, or incomplete features. To this aim, we analyzed a statistically significant sample of 443 SATD comments found in 54 projects dependent on TensorFlow or PyTorch.

As a result, we elicited a taxonomy of 41 DL-specific SATD types, organized into seven categories, in turn further grouped into two high-level categories related to Infrastructure and DL Life-Cycle. We discussed the implications of the found SATDs, as well as the differences with previously-studied DL bugs, and the inadequacy of popular Python static analysis tools to cope with such SATDs.

In future work, we plan to complement this study with further investigations with practitioners, and to develop approaches able to detect and repay DL-specific SATD.

Acknowledgments

Massimiliano Di Penta acknowledges the Italian PRIN 2020 Project EMELIOT "Engineered MachinE Learning-intensive IoT systems", ID 2020W3A5FY. Federica Pepe is partially funded by the PNRR DM 352/2022 Italian Grant for Ph.D. scholarships. Fiorella Zampetti is partially funded by the PON DM 1062/2021.

References

[1] *Sixth International Workshop on Managing Technical Debt, MTD@ICSME 2014, Victoria, BC, Canada, September 30, 2014*. IEEE Computer Society, 2014. [Online]. Available: https://ieeexplore.ieee.org/xpl/conhome/6970370/proceeding

[2] A. Ahmad, A. Tariq, H. K. Hussain, and A. Y. Gill, "Revolutionizing healthcare: How deep learning is poised to change the landscape of medical diagnosis and treatment," *Journal of Computer Networks, Architecture and High Performance Computing*, vol. 5, no. 2, pp. 458–471, 2023.

[3] Y. Alghofaili, A. Albattah, and M. A. Rassam, "A financial fraud detection model based on LSTM deep learning technique," *Journal of Applied Security Research*, vol. 15, no. 4, pp. 498–516, 2020.

[4] G. Bavota and B. Russo, "A large-scale empirical study on self-admitted technical debt," in *International Conference on Mining Software Repositories*, M. Kim, R. Robbes, and C. Bird, Eds. ACM, 2016, pp. 315–326.

[5] A. Bhat, A. Coursey, G. Hu, S. Li, N. Nahar, S. Zhou, C. Kästner, and J. L. C. Guo, "Aspirations and practice of ML model documentation: Moving the needle with nudging and traceability," in *CHI*. ACM, 2023, pp. 749:1–749:17.

[6] A. Bhatia, F. Khomh, B. Adams, and A. E. Hassan, "An empirical study of self-admitted technical debt in machine learning software," *arXiv preprint arXiv:2311.12019*, 2023.

[7] J. Bogner, R. Verdecchia, and I. Gerostathopoulos, "Characterizing technical debt and antipatterns in AI-based systems: A systematic mapping study," in *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*. IEEE, 2021, pp. 64–73.

[8] N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya *et al.*, "Managing technical debt in software-reliant systems," in *Proceedings of the FSE/SDP workshop on Future of software engineering research*, 2010, pp. 47–52.

[9] J. Chen, Y. Liang, Q. Shen, J. Jiang, and S. Li, "Toward understanding deep learning framework bugs," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, pp. 1–31, 2023.

[10] P. S. Chib and P. Singh, "Recent advancements in end-to-end autonomous driving using deep learning: A survey," *IEEE Transactions on Intelligent Vehicles*, 2023.

[11] N. A. Ernst, S. Bellomo, I. Ozkaya, R. L. Nord, and I. Gorton, "Measure it? manage it? ignore it? software practitioners and technical debt," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 50–60.

[12] Google Inc., "Keras," 2024. [Online]. Available: https://keras.io

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.

[13] Q. Guo, J. Cao, X. Xie, S. Liu, X. Li, B. Chen, and X. Peng, "Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3623306

[14] J. Han, E. Shihab, Z. Wan, S. Deng, and X. Xia, "What do programmers discuss about deep learning frameworks," *Empir. Softw. Eng.*, vol. 25, no. 4, pp. 2694–2747, 2020.

[15] S. C. Y. Ho, V. Majdinasab, M. Islam, D. E. Costa, E. Shihab, F. Khomh, S. Nadi, and M. Raza, "An empirical study on bugs inside pytorch: A replication study," in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2023, pp. 220–231.

[16] Hugging Face Inc., "Hugging Face," 2024. [Online]. Available: https://huggingface.co

[17] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1110–1121.

[18] L. Jia, H. Zhong, X. Wang, L. Huang, and X. Lu, "The symptoms, causes, and repairs of bugs inside a deep learning library," *Journal of Systems and Software*, vol. 177, p. 110935, 2021.

[19] P. Kruchten, R. L. Nord, I. Ozkaya, and D. Falessi, "Technical debt: towards a crisper definition report on the 4th international workshop on managing technical debt," *ACM SIGSOFT Software Engineering Notes*, vol. 38, no. 5, pp. 51–54, 2013.

[20] Landscape, "Prospector," 2024. [Online]. Available: https://github.com/landscapeio/prospector

[21] Y. Li, M. Soliman, and P. Avgeriou, "Automatic identification of self-admitted technical debt from four different sources," *Empirical Software Engineering*, vol. 28, no. 3, p. 65, 2023.

[22] E. Lim, N. Taksande, and C. Seaman, "A balancing act: What software practitioners have to say about technical debt," *IEEE software*, vol. 29, no. 6, pp. 22–27, 2012.

[23] J. Liu, Q. Huang, X. Xia, E. Shihab, D. Lo, and S. Li, "Is using deep learning frameworks free? characterizing technical debt in deep learning frameworks," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Society*, 2020, pp. 1–10.

[24] ——, "An exploratory study on the introduction and removal of different types of technical debt in deep learning frameworks," *Empirical Software Engineering*, vol. 26, pp. 1–36, 2021.

[25] Logilab and Pylint contributors, "Pylint," 2024. [Online]. Available: https://pylint.pycqa.org/

[26] E. Maldonado and E. Shihab, "Detecting and quantifying different types of self-admitted technical debt," in *2015 IEEE 7Th international workshop on managing technical debt (MTD)*. IEEE, 2015, pp. 9–15.

[27] J. P. Meher, S. Biswas, and R. Mall, "Deep learning-based software bug classification," *Information and Software Technology*, vol. 166, p. 107350, 2024.

[28] MILA, "Theano," 2017. [Online]. Available: https://github.com/Theano/Theano

[29] M. M. Morovati, A. Nikanjam, F. Tambon, F. Khomh, and Z. M. Jiang, "Bug characterization in machine learning-based systems," *Empirical Software Engineering*, vol. 29, no. 1, p. 14, 2024.

[30] MyPy, "MyPy," 2024. [Online]. Available: https://mypy-lang.org

[31] Nirjas, "Nirjas Python library https://github.com/fossology/Nirjas," 2023, (Last access: 26/02/2024).

[32] D. OBrien, S. Biswas, S. Imtiaz, R. Abdalkareem, E. Shihab, and H. Rajan, "23 shades of self-admitted technical debt: an empirical study on machine learning software," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 734–746.

[33] F. Pepe, F. Zampetti, A. Mastropaolo, G. Bavota, and M. Di Penta, "Replication Package of the paper: "A Taxonomy of Self-Admitted Technical Debt in Deep Learning Systems"," Apr. 2024. [Online]. Available: https://doi.org/10.5281/zenodo.10958661

[34] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 2014, pp. 91–100.

[35] L. Rantala, M. Mäntylä, and V. Lenarduzzi, "Keyword-labeled self-admitted technical debt and static code analysis have significant relationship but limited overlap," *Software Quality Journal*, pp. 1–39, 2023.

[36] L. Rantala, M. Mäntylä, and D. Lo, "Prevalence, contents and automatic detection of kl-satd," in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2020, pp. 385–388.

[37] X. Ren, Z. Xing, X. Xia, D. Lo, X. Wang, and J. Grundy, "Neural network-based detection of self-admitted technical debt: From performance to explainability," *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 3, p. 15, 2019. [Online]. Available: https://doi.org/10.1145/3324916

[38] L. Richardson, "Beautifulsoup," https://www.crummy.com/software/BeautifulSoup/, 2024.

[39] B. Rosner, *Fundamentals of Biostatistics*, 7th ed. Brooks/Cole, Boston, MA, 2011.

[40] C. Seaman and Y. Guo, "Measuring and monitoring technical debt advances in computers," 2011.

[41] D. Spencer, *Card sorting: Designing usable categories*. Rosenfeld Media, 2009.

[42] I. Stapleton Cordasco, "Flake8," 2024. [Online]. Available: https://flake8.pycqa.org/en/latest/

[43] F. Tambon, A. Nikanjam, L. An, F. Khomh, and G. Antoniol, "Silent bugs in deep learning frameworks: An empirical study of keras and tensorflow," *Empirical Software Engineering*, vol. 29, no. 1, p. 10, 2024.

[44] Tesla, "News https://www.newsweek.com/autonomous-tesla-crashes-parked-fire-truck-california-freeway-789177," 2018, (Last access: 26/03/2024).

[45] Uber, "News https://www.vice.com/en/article/9kga85/uber-is-giving-up-on-self-driving-cars-in-california-after-deadly-crash," 2018, (Last access: 26/03/2024).

[46] C. Watson, N. Cooper, D. N. Palacio, K. Moran, and D. Poshyvanyk, "A systematic literature review on the use of deep learning in software engineering research," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 2, pp. 1–58, 2022.

[47] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," 2023.

[48] L. Xavier, F. Ferreira, R. Brito, and M. T. Valente, "Beyond the code: Mining self-admitted technical debt in issue tracker systems," in *Proceedings of the 17th international conference on mining software repositories*, 2020, pp. 137–146.

[49] Y. Yang, T. He, Z. Xia, and Y. Feng, "A comprehensive empirical study on bug characteristics of deep learning frameworks," *Information and Software Technology*, vol. 151, p. 107004, 2022.

[50] F. Zampetti, G. Fucci, A. Serebrenik, and M. Di Penta, "Self-admitted technical debt practices: a comparison between industry and open-source," *Empirical Software Engineering*, vol. 26, pp. 1–32, 2021.

[51] N. Zazworka, M. A. Shaw, F. Shull, and C. Seaman, "Investigating the impact of design debt on software quality," in *Proceedings of the 2nd workshop on managing technical debt*, 2011, pp. 17–23.

[52] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang, "An empirical study on tensorflow program bugs," in *Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis*, 2018, pp. 129–140.

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 13,2026 at 10:34:42 UTC from IEEE Xplore. Restrictions apply.